ML2 - Assignment 1

Linear Regression & Generalization

1. Overview

In this assignment you will train a linear regression model on a dataset with 231 features and 500 training samples. Your goal is to learn a weight vector w such that the prediction y = X @ w generalizes well to unseen data.

This assignment is designed to give you hands-on experience with key concepts in machine learning: overfitting, regularization, feature selection, and the gap between training performance and real-world performance.

2. Data

You are provided with a single file:

- **train.csv -** 500 samples, 231 features (x0 through x230) and a target column (y).

The features include a mix of raw measurements, transformed variables, and a constant (bias) column. Not all features are informative - some are noise that will hurt your model if included without regularization.

3. Task

Train a linear regression model using Stochastic Gradient Descent (SGD):

```
y = X @ w
```

Your deliverable is a weight vector w of length 231, saved as a NumPy .npy file. Upload this file to the class leaderboard to see your score.

4. Leaderboard

The leaderboard is hosted at:

```
https://ml2-homework-server.onrender.com
```

When you submit your weight vector, the server evaluates it on three datasets:

- **Train MSE -** Your model's error on the training data you already have. A low train MSE does not guarantee good generalization.
- **Client MSE -** Your model's error on a held-out client sample. This is the metric used to rank submissions on the leaderboard.
- **Live MSE -** Your model's error on a separate live dataset, revealed only after the deadline. This simulates real-world deployment where you cannot tune against the test set.

> The gap between Train MSE and Client MSE tells you how much your model is overfitting. The gap between Client MSE and Live MSE (revealed later) tells you how well your model truly generalizes.

5. Rules

- You may only use linear models: y = X @ w (no neural networks, trees, etc.).
- You must implement or use SGD for optimization.
- You may use any feature selection, regularization, or cross-validation strategy.
- You may use NumPy, pandas, matplotlib, and scikit-learn.
- Your submission is a single .npy file containing a weight vector of length 231.

6. How to Submit

1. Train your model and obtain a weight vector w of shape (231,).
2. Save it as a .npy file:

```
np.save('weights.npy', w)
```

3. Go to the leaderboard website, fill in your name, upload the .npy file, and click Submit.

7. Starter Code

You are provided with:

- **starter_kit.py -** A starting point with data loading, SGD implementation, standardization helpers, and cross-validation utilities.

The starter kit achieves a reasonable score but is far from optimal. Your job is to beat it.

8. Hints

- Standardize your features before training with SGD (but be careful with the constant/bias column - do not standardize it).
- Try both L1 (Lasso) and L2 (Ridge) regularization. What happens to the weights?
- Use cross-validation to choose hyperparameters (learning rate, regularization strength, number of epochs).
- When converting weights from standardized space to raw space for submission, make sure the intercept is handled correctly.
- A model with very low training MSE but high client MSE is overfitting. Adding regularization or removing features can help.

Good luck! Build models that generalize.